1. What is Machine Learning ?

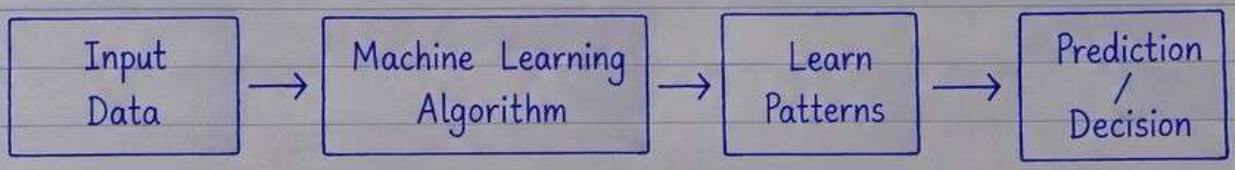
1.1 Definition :

Machine Learning (ML) is a subset of Artificial Intelligence that enables computers to learn patterns from data without being explicitly programmed.

1.2 Key Points :

- ML uses algorithms to learn from Historical data.
- It automatically finds patterns and makes predictions or decisions.
- It improves its performance with more data and experience.
- Widely used in real-world applications.

1.3 How Machine Learning Works ?



1.4 Example :

- Email spam detection.
- House price prediction.
- Image recognition.
- Recommendation systems (YouTube, Netflix).

1.5 Types of Problems :

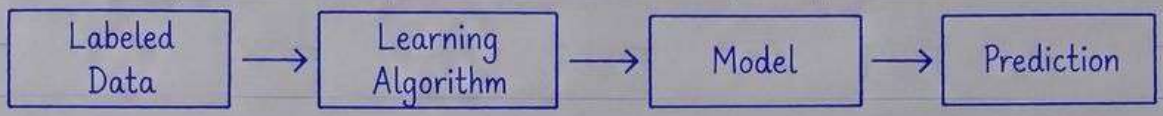
- Classification (e.g., spam or not :)
- Regression (e.g., predict house price)
- Clustering (e.g., group similar customers)
- Anomaly Detection (e.g., fraud detection)

2. Types of Machine Learning

Machine Learning can be broadly classified into four main types based on the type of data and learning approach.

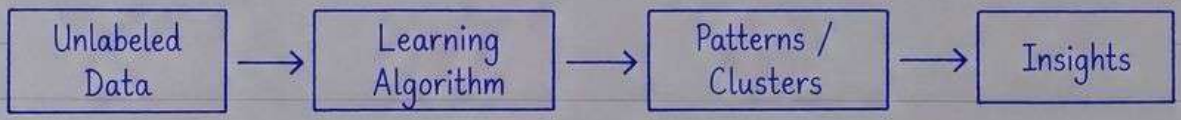
2.1 Supervised Learning :

- Uses labeled data (input and output both are known).
- Learns a mapping from input to output.
- Used for classification and regression problems.
- Examples : Email spam detection, house price prediction.



2.2 Unsupervised Learning :

- Uses unlabeled data (only input data is available).
- Finds hidden patterns or structure in the data.
- Used for clustering, dimensionality reduction, etc.
- Examples : Customer segmentation, grouping similar products.



2.3 Reinforcement Learning :

- Learns by interacting with an environment.
- Receives rewards or penalties for actions.
- Goal is to learn a policy that maximizes total reward.
- Examples : Game playing (chess, Go), robotics, self-driving cars.



2.4 Semi - Supervised Learning :

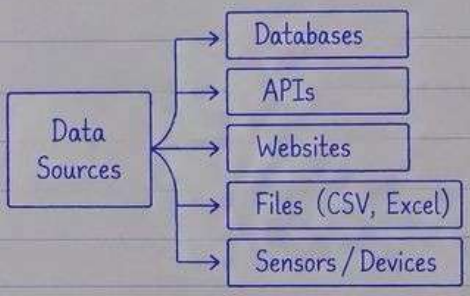
- Uses a combination of labeled and unlabeled data.
- Useful when labeled data is limited and unlabeled data is large.
- Combines the advantages of supervized and unsupervised learning.
- Examples : Image classification, text classification.

3. Data Collection and Understanding

Data collection is the process of gathering relevant data from different sources and understanding its structure, quality and meaning before building a machine learning model.

3.1 Sources of Data :

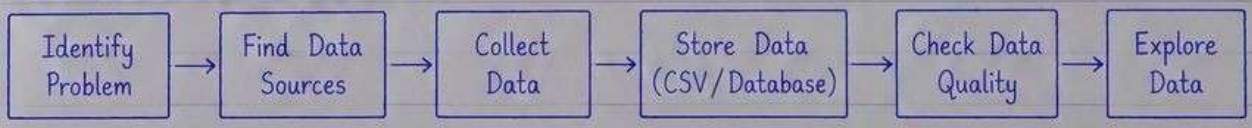
- Internal data (company database, application data)
- External data (public datasets, APIs, websites)
- Manual collection (surveys, forms, sensors)
- Open source datasets (Kaggle, UCI, Govt. portals)



3.2 Types of Data :

- Structured Data : Tabular format (rows and columns)
- Unstructured Data : Text, images, audio, video
- Semi-structured Data : JSON, XML, logs
- Labeled Data : Contains target/output variable
- Unlabeled Data : No target/output variable

3.3 Data Collection Process :



3.4 Common Data Formats :

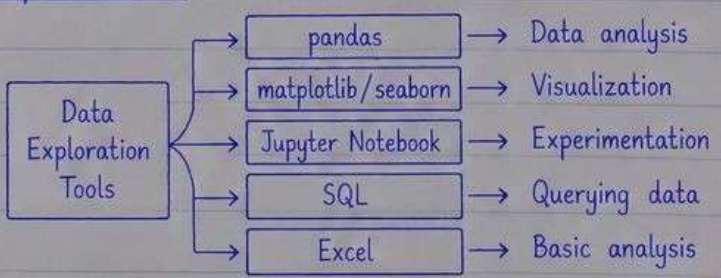
Format	Description	Example Use
CSV	Simple text format, widely used	Tabular data (small to medium)
Excel	Easy to use, good for small datasets	Business data, manual collection
JSON	Common in web and API data	API responses, semi-structured data
SQL	Used in databases	Large structured data
Parquet	Efficient format for large data	Big data and data lakes

3.5 Understanding the Data :

- Look at the shape of the data (rows and columns).
- Check the data types of each column.
- Identify missing values.
- Understand the meaning of each feature.
- Find basic statistics (mean, median, min, max).
- Visualize the data to find patterns and outliers.

3.6 Tools for Data Collection and Exploration :

- Python (pandas, numpy)
- Jupyter Notebook
- SQL (for database data)
- Excel (small datasets)
- APIs (requests library)
- Web scraping (BeautifulSoup, Scrapy)
- Visualization (matplotlib, seaborn)



4. Data Preprocessing

Data preprocessing is an important step in Machine Learning. It helps to clean, transform and prepare raw data into a suitable format so that models can learn better and give accurate results.

4.1 Need for Data Preprocessing :

- Real world data is often noisy, incomplete and inconsistent.
- Improves data quality and model performance.
- Handles missing values, duplicates and outliers.
- Converts data into a suitable format for ML algorithms.

4.2 Common Steps in Data Preprocessing :



4.3 Handling Missing Values :

- Data may have missing or null values.
- Methods to handle missing values :
  - Remove rows or columns (if less data)
  - Fill with mean / median / mode
  - Use constant value (e.g. 0)
  - Use forward / backward fill (for time series)

Method	Example (Pandas)
Mean	df.fillna(df['col'].mean())
Median	df.fillna(df['col'].median())
Mode	df.fillna(df['col'].mode()[0])
Constant	df.fillna(0)
Forward Fill	df.fillna(method='ffill')
Backward Fill	df.fillna(method='bfill')

4.4 Handling Duplicates :

- Duplicate rows can affect analysis and model performance.
- We can remove duplicates using pandas.

```
df.drop_duplicates(inplace = True)
```

Before

Id	Name	Age
1	Rahul	28
1	Rahul	28
2	Priya	25

→

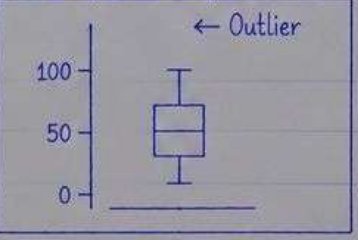
After

Id	Name	Age
1	Rahul	28
2	Priya	25

4.5 Handling Outliers :

- Outliers are extreme values that can negatively impact the model.
- Methods :
  - Visualize using boxplot
  - Remove outliers
  - Capping (replace with threshold)
  - Use robust scaling

Boxplot Example :



4.6 Encoding Categorical Data :

- Machine learning algorithms work with numerical data, so we convert categorical data into numbers.
- Types of Encoding :
  - Label Encoding (convert categories to numbers)
  - One-Hot Encoding (create binary columns)

Label Encoding Example :

City	Encoded
Delhi	0
Mumbai	1
Bangalore	2

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of analyzing and visualizing data to understand its main characteristics, find patterns, detect anomalies and get insights before building a machine learning model.

5.1 Goals of EDA :

- Understand the structure of the data.
- Find missing values and outliers.
- Analyze distributions of features.
- Find relationships between variables.
- Discover patterns and trends.
- Get insights for feature engineering.
- Make better decisions for model building.

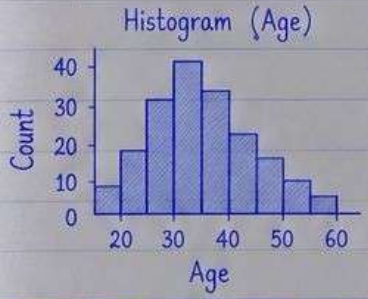
5.2 Summary Statistics :

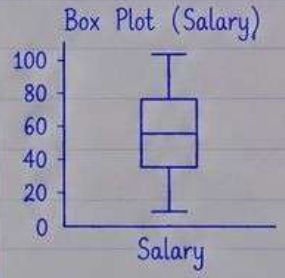
- Helps to understand the central tendency, spread and shape of the data.
- Common statistics :

Measure	Meaning
Count	Total number of values
Mean	Average value
Median	Middle value
Std Deviation	How much data is spread
Min / Max	Smallest and largest value
Percentiles	Value below which a % of data falls

5.3 Univariate Analysis :

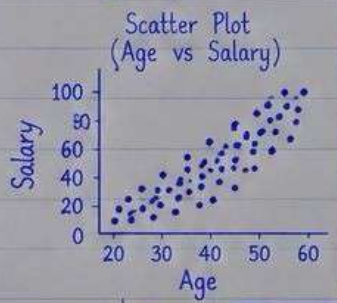
- Analyze each feature individually.
- Understand the distribution, range and outliers.
- Common plots : Histogram, Box Plot, KDE Plot.





5.4 Bivariate Analysis :

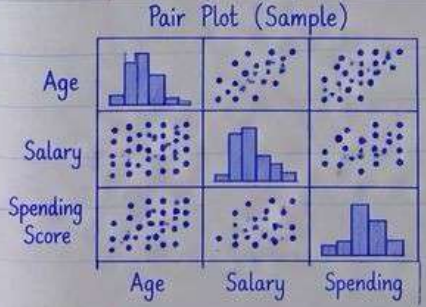
- Analyze the relationship between two variables.
- Used to find correlation, trends and patterns.
- Common plots : Scatter Plot, Line Plot, Bar Plot.

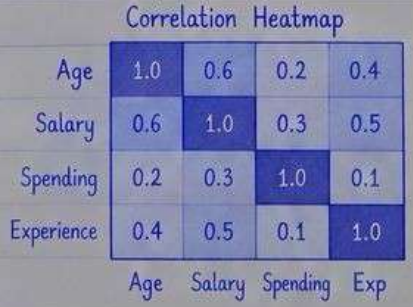




5.5 Multivariate Analysis :

- Analyze multiple variables together.
- Helps to find complex relationships and patterns.
- Common plots : Pair Plot, Heatmap.





5.6 Using Pandas for EDA :

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("data.csv")
# Summary statistics
df.describe()
# Check missing values
df.isnull().sum()
# Correlation matrix
df.corr()
# Plot histogram
df['Age'].plot(kind='hist')
# Plot boxplot
df.boxplot(column=["Salary"])
# Plot heatmap
sns.heatmap(df.corr(), annot=True)
```

5.7 Key Insights from EDA :

- Identify important features.
- Detect and handle outliers.
- Find correlations between variables.
- Understand the distribution and skewness.
- Get ideas for feature engineering.
- Helps in selecting the right algorithm.

5.8 Popular Visualization Libraries :

Library	Use
Matplotlib	Basic plots (histogram, scatter, etc.)
Seaborn	Advanced statistical visualizations
Plotly	Interactive visualizations
Pandas	Quick and easy plots
Missingno	Visualize missing values

6. Machine Learning Algorithms

Machine Learning algorithms are the core models that learn from data and make predictions or decisions. Here are the most important algorithms.

6.1 Linear Regression :

- Used for predicting continuous values.
- Finds the best fit line ($y = mx + b$).
- Minimizes Mean Squared Error (MSE).
- Example : Predict house price, sales.

Equation :

$$y = mx + b$$

(Simple Linear Regression)

6.2 Logistic Regression :

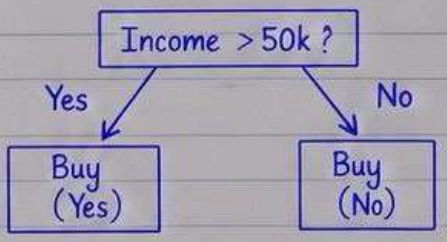
- Used for classification problems.
- Predicts probability (0 to 1) using sigmoid.
- Works for binary and multi-class classification.
- Example : Spam detection, disease prediction.

Sigmoid Function :

$$p = \frac{1}{1 + e^{-z}}$$

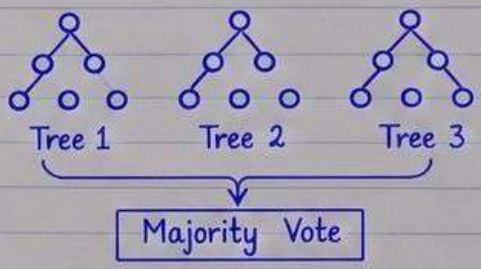
6.3 Decision Tree :

- Tree-based model for classification & regression.
- Uses feature conditions (if-else rules).
- Easy to interpret and visualize.
- Example : Loan approval, customer churn.



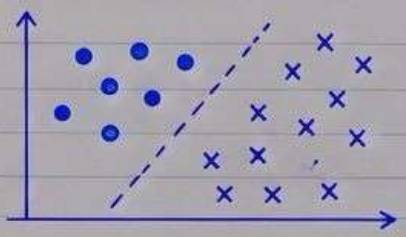
6.4 Random Forest :

- Ensemble of multiple decision trees.
- Improves accuracy and reduces overfitting.
- Works for classification and regression.
- Example : Disease prediction, fraud detection.



6.5 Support Vector Machine (SVM) :

- Finds the best hyperplane to separate classes.
- Works well in high-dimensional space.
- Uses kernel trick for non-linear data.
- Example : Image classification, text classification.



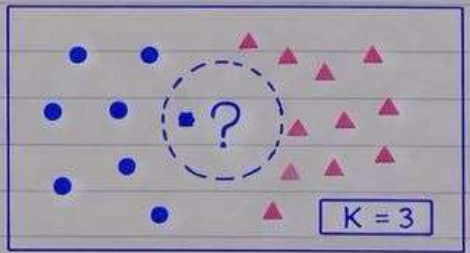
6.6 Naive Bayes :

- Based on Bayes theorem and feature independence.
- Works well for text and categorical data.
- Fast and scalable.
- Example : Spam filter, document classification.

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

6.7 K-Nearest Neighbors (KNN) :

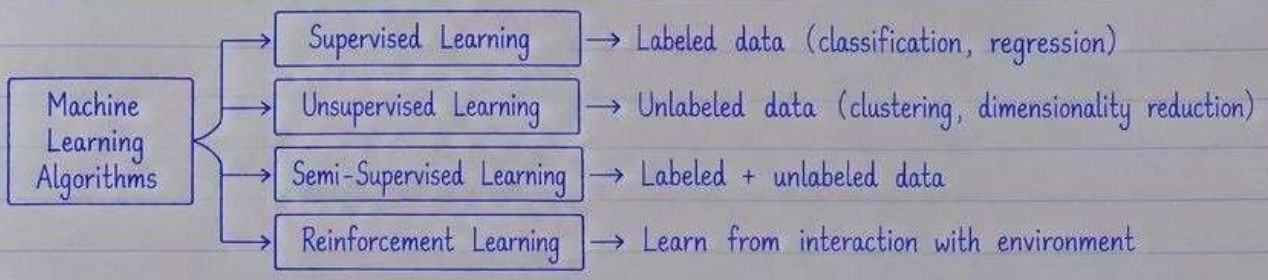
- Simple instance-based learning algorithm.
- Predicts based on k nearest data points.
- Works for classification and regression.
- Example : Handwritten digit recognition.



7. Common Machine Learning Algorithms

Machine Learning algorithms are of different types and are used to solve different kinds of problems. The choice of algorithm depends on the type of data, problem statement and expected output.

7.1 Types of Machine Learning Algorithms :



7.2 Supervised Learning Algorithms :

- Uses labeled data (input + output).
- Divided into Classification and Regression.
- Learns a mapping from input to output.
- Common algorithms :

Algorithm	Use Case
Linear Regression	Predict continuous values
Logistic Regression	Binary/multi-class classification
Decision Tree	Classification and regression
Random Forest	Improves decision tree performance
Support Vector Machine (SVM)	Classification with optimal boundary
K-Nearest Neighbors (KNN)	Classification and regression
Naive Bayes	Text classification, spam detection

7.3 Unsupervised Learning Algorithms :

- Uses unlabeled data (only input).
- Finds patterns and structure in data.
- Common algorithms :

Algorithm	Use Case
K-Means Clustering	Group similar data points
Hierarchical Clustering	Builds a hierarchy of clusters
DBSCAN	Finds clusters with noise
PCA (Principal Component Analysis)	Dimensionality reduction
t-SNE	Visualize high-dimensional data

7.4 Semi-Supervised Learning :

- Uses a small amount of labeled data and a large amount of unlabeled data.
- Useful when labeling data is expensive.
- Common algorithms :

Algorithm	Use Case
Self-Training	Uses model predictions as labels
Label Propagation	Spreads labels to similar data points
Semi-Supervised SVM	Classification with limited labels

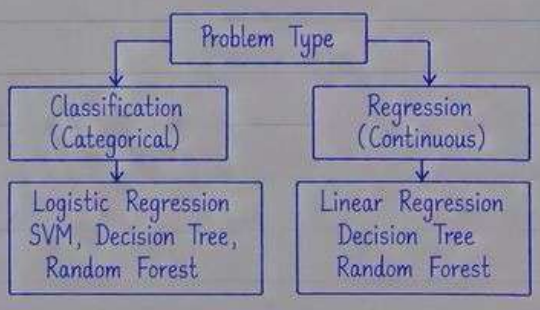
7.5 Reinforcement Learning :

- Learns through interaction with environment.
- Receives a reward or penalty for actions.
- Goal is to maximize cumulative reward.
- Common algorithms :

Algorithm	Use Case
Q-Learning	Learn optimal action policy
Deep Q-Network (DQN)	Uses neural networks
Policy Gradient	Learns policy directly
Actor-Critic	Combines value and policy based methods

7.6 How to Choose the Right Algorithm :

- If output is categorical → Use Classification algorithms.
- If output is continuous → Use Regression algorithms.
- If data is unlabeled → Use Clustering or Dimensionality Reduction.
- If you have sequential decision making → Use Reinforcement Learning.
- Consider dataset size, features, interpretability and performance.



8. Regression Algorithms

Regression is a supervised learning algorithm used to predict continuous numerical values. It finds the relationship between independent variables (features) and a dependent variable (target) by learning from training data.

8.1 Types of Regression Algorithms :

- Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression
- Support Vector Regression (SVR)
- Decision Tree Regression
- Random Forest Regression
- Gradient Boosting Regression (XGBoost, LightGBM)

8.2 Linear Regression :

- Assumes a linear relationship between input features and target.
- Simple and easy to implement.
- Works well when the relationship is linear.
- Model equation :

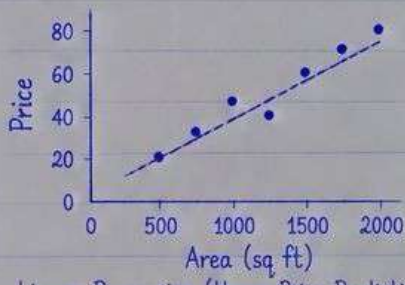
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

- β_0 : Intercept
- $\beta_1, \beta_2 \ldots$: Coefficients

8.3 Example : Simple Linear Regression

- Predict house price based on area.

Area (sq ft)	Price (in Lakhs)
500	20
1000	35
1500	50
2000	70



Linear Regression (House Price Prediction)

Python Implementation :

```
from sklearn.linear_model import LinearRegression
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[500],[1000],[1500],[2000]])
y = np.array([20, 35, 50, 70])
model = LinearRegression()
model.fit(X, y)
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

8.4 Multiple Linear Regression :

- Uses multiple independent variables.
- Model equation :

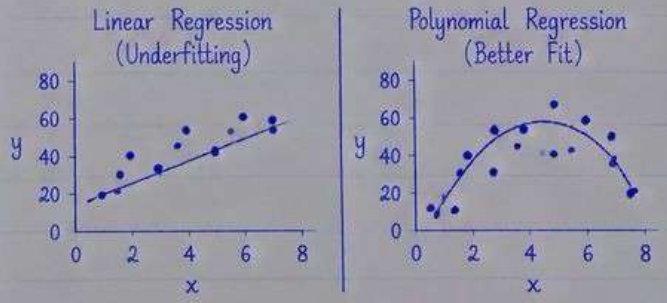
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Example : Predict house price using area, bedrooms and age.

Area	Bedrooms	Age	Price
1000	2	5	40
1500	3	3	55
2000	4	2	70

8.5 Polynomial Regression :

- Used when the relationship is non-linear.
- Transforms input features using polynomial terms.
- Example : $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$



8.6 Regularization in Regression :

- Helps to prevent overfitting by adding a penalty term.
- Types :
  - Ridge Regression (L2 Regularization)
  - Lasso Regression (L1 Regularization)
  - Elastic Net (Combination of L1 and L2)

8.7 Evaluation Metrics :

Metric	Formula	Use Case
MAE	$\frac{1}{n}\sum_{i=1}^{n} \lvert y_i - \hat{y}_i \rvert$	Average error (robust to outliers)
MSE	$\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$	Penalizes large errors
RMSE	$\sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$	More interpretable (same unit as target)
R^2 Score	$1 - \frac{SS_{res}}{SS_{tot}}$	Explains variance (values between 0 and 1)

9. Classification Algorithms

Classification is a supervised learning technique used to predict a categorical output (class label) based on input features. It learns a decision boundary from training data to classify new, unseen data into predefined classes.

9.1 Common Classification Algorithms :

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Naive Bayes
- Gradient Boosting (XGBoost, LightGBM)
- Neural Network (for complex problems)

9.2 | Logistic Regression :

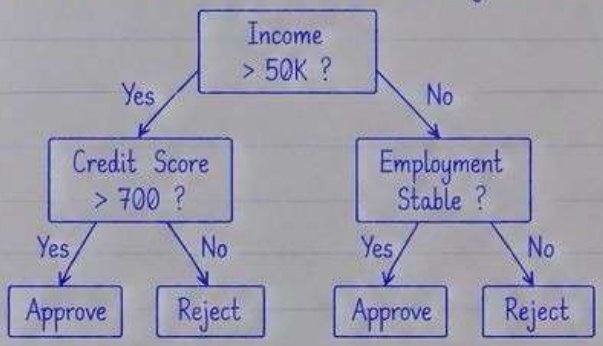
- Used for binary and multi-class classification.
- Predicts probability using Sigmoid function.
- Model equation :

$$P(y=1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)}}$$

- If $P(y=1) \geq 0.5 \rightarrow$ Class 1
- If $P(y=1) < 0.5 \rightarrow$ Class 0

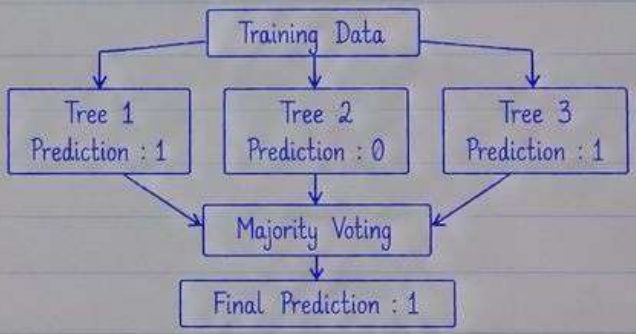
9.3 Decision Tree :

- Splits the data based on feature values.
- Forms a tree-like structure.
- Easy to interpret and visualize.
- Can handle both numerical and categorical data.



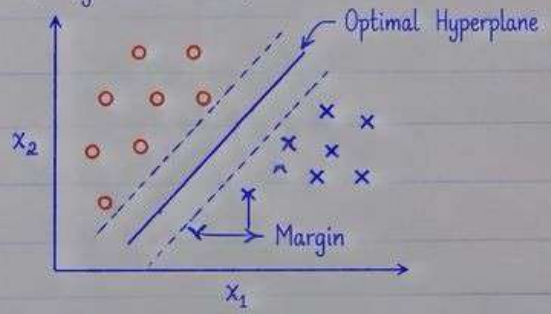
9.4 | Random Forest :

- Ensemble of multiple decision trees.
- Uses bagging (bootstrap aggregating).
- Reduces overfitting and improves accuracy.
- Final prediction is based on majority voting (for classification).



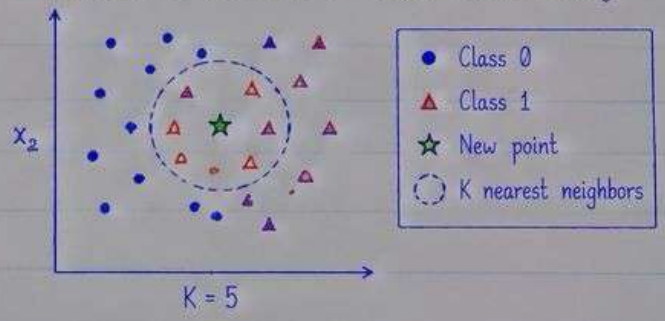
9.5 Support Vector Machine (SVM) :

- Finds the best hyperplane to separate classes.
- Works well in high dimensional space.
- Useful for both linear and non-linear data (using kernel trick).



9.6 | K-Nearest Neighbors (KNN) :

- Classifies based on majority class of K nearest points.
- Simple and easy to implement.
- Sensitive to the value of K and feature scaling.



9.7 Naive Bayes :

- Based on Bayes Theorem.
- Assumes independence between features (naive assumption).
- Works well for text classification (spam detection).

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

9.8 | Model Evaluation for Classification :

Metric	Formula / Description	Use Case
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall performance
Precision	$TP / (TP + FP)$	Minimize false positives
Recall	$TP / (TP + FN)$	Minimize false negatives
F1-Score	$2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$	Balances precision and recall
Confusion Matrix	TP, TN, FP, FN	Detailed evaluation

10. Clustering Algorithms

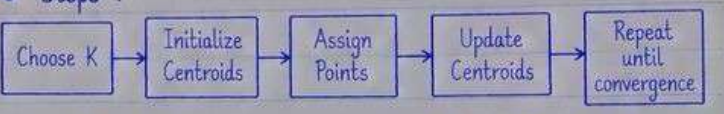
Clustering is an unsupervised learning technique used to group similar data points together based on their features. It finds hidden patterns in data without using labeled outputs.

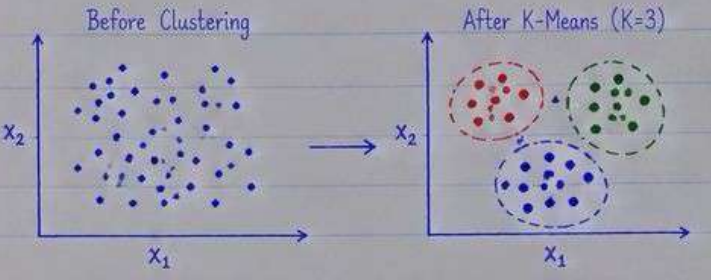
10.1 Common Clustering Algorithms :

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN (Density Based)
- Gaussian Mixture Model (GMM)
- Mean Shift
- Spectral Clustering
- Agglomerative and Divisive Clustering

10.2 | K-Means Clustering :

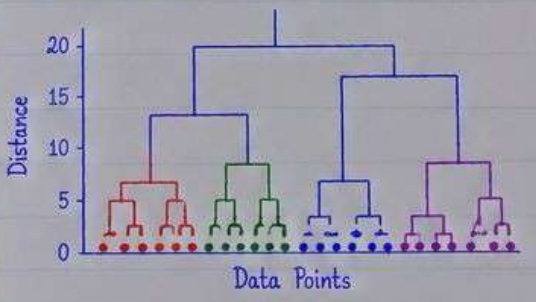
- Divides data into K clusters.
- Each point is assigned to the nearest centroid.
- Iteratively updates centroids.
- Goal : Minimize within-cluster sum of squares (WCSS).
- Steps :





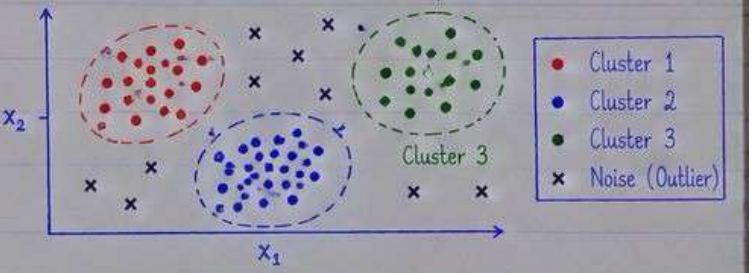
10.3 Hierarchical Clustering :

- Builds a hierarchy of clusters (dendrogram).
- Two approaches :
  - Agglomerative (Bottom-Up) : Start with single points and merge clusters.
  - Divisive (Top-Down) : Start with all points and split clusters.
- Distance metrics : Euclidean, Manhattan, Cosine, etc.



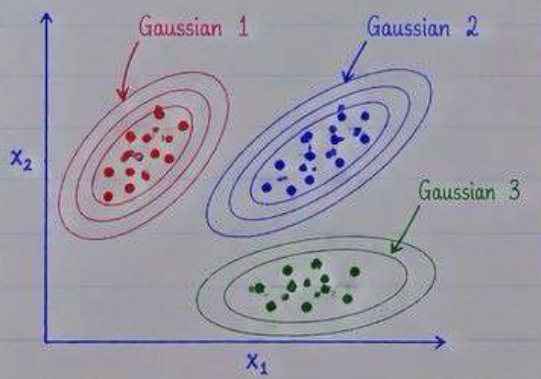
10.4 | DBSCAN (Density Based Clustering) :

- Groups points that are closely packed together.
- Can find clusters of arbitrary shape.
- Handles noise (outliers) well.
- Parameters :
  - eps : Maximum distance between two points to be considered neighbors.
  - minPts : Minimum number of points to form a cluster.



10.5 Gaussian Mixture Model (GMM) :

- Assumes data is generated from a mixture of multiple Gaussian distributions.
- Each cluster is a Gaussian with its own mean and covariance.
- Uses Expectation-Maximization (EM) algorithm.
- Soft clustering (probability of belonging to a cluster).
- Useful when clusters are overlapping.



10.6 | Choosing the Right Clustering Algorithm :

Algorithm	Best Use Case	Advantages	Limitations
K-Means	Spherical clusters	Simple, fast	Sensitive to outliers, requires K
Hierarchical	Small datasets	No need to specify K	Computationally expensive
DBSCAN	Arbitrary shapes, noise	Handles outliers	Struggles with varying density
GMM	Overlapping clusters	Probabilistic clustering	Assumes Gaussian distribution

10.7 | Evaluation Metrics for Clustering :

- Silhouette Score : Measures how similar a point is to its own cluster compared to other clusters. (Range : -1 to 1)
- Davies-Bouldin Index : Lower value is better.
- Calinski-Harabasz Index : Higher value is better.
- Elbow Method : Helps to find the optimal number of clusters (K).

SUPPORT VECTOR MACHINE (SVM)

1. What is SVM ?

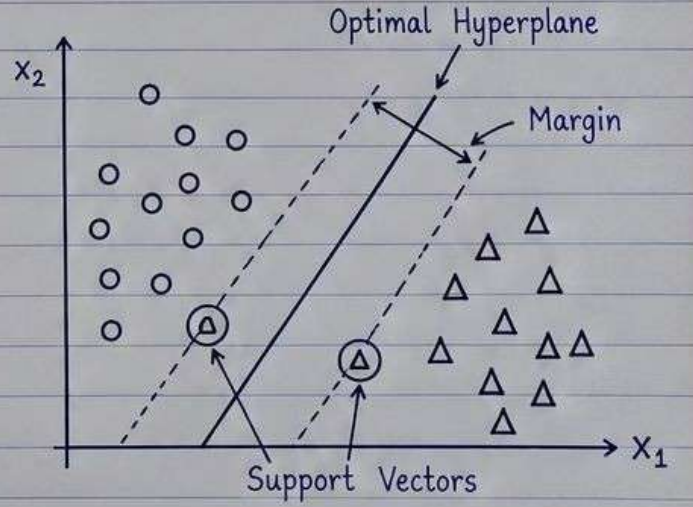
- SVM (Support Vector Machine) is a supervised machine learning algorithm used for classification and regression (SVR).
- It works by finding the optimal hyperplane that best separates the classes in the feature space.
- SVM tries to maximize the margin between the closest data points (support vectors) of different classes.

2. Key Concepts

- Hyperplane : A decision boundary that separates classes.
- Margin : Distance between hyperplane and nearest data points.
- Support Vectors : Data points that are closest to the hyperplane.
- Kernel Trick : Helps to handle non-linearly separable data.
- Soft Margin : Allows some misclassification using a penalty.

3. How SVM Works ?

1. Transform data into higher dimension (if needed).
2. Find the optimal hyperplane with maximum margin.
3. Use support vectors to define the hyperplane.
4. Classify new data based on the hyperplane.



4. Types of SVM

- Linear SVM : Used when data is linearly separable.
- Non-Linear SVM : Used with Kernel Trick for complex data.
- Support Vector Regression (SVR) : Used for regression problems.

5. Kernel Functions

Kernel	Formula	Use Case
Linear	$K(x, x') = x \cdot x'$	Linearly separable data
Polynomial	$K(x, x') = (\gamma x \cdot x' + r)^d$	Non-linear data (polynomial)
RBF (Gaussian)	$K(x, x') = \exp(-\gamma \lVert x - x' \rVert^2)$	Most popular, works well in most cases
Sigmoid	$K(x, x') = \tanh(\gamma x \cdot x' + r)$	Neural network like

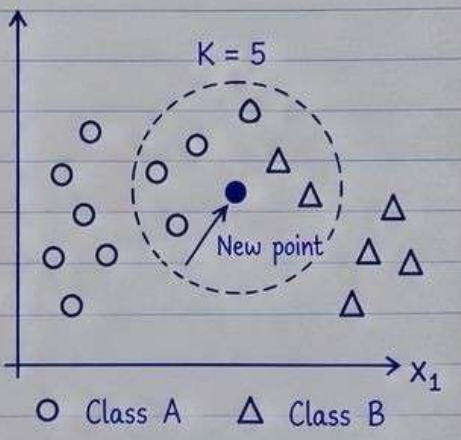
K - NEAREST NEIGHBORS (KNN)

1. What is KNN ?

- K-Nearest Neighbors (KNN) is a simple, instance-based machine learning algorithm used for classification and regression.
- It classifies a new data point based on the majority class of its k nearest neighbors.
- It is a non-parametric and lazy learning algorithm (no training phase).
- It works based on the distance between data points in the feature space.
- KNN can be used for both classification and regression problems.

2. How KNN Works ?

1. Choose the value of K (number of neighbors).
2. Calculate the distance between the new data point and all training data points.
3. Select the K nearest neighbors.
4. For classification: choose the class with majority vote.
5. For regression: take the average of target values of the K neighbors.



3. Distance Metrics

- Euclidean Distance : Most commonly used.
- Manhattan Distance : Works well with high dimensional data.
- Minkowski Distance : General form (combination of Euclidean and Manhattan).

Euclidean Distance	Manhattan Distance	Minkowski Distance
$d(x,y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$	$d(x,y) = \sum_{i=1}^{n} \lvert x_i - y_i \rvert$	$d(x,y) = \left(\sum_{i=1}^{n} \lvert x_i - y_i \rvert^p \right)^{1/p}$

4. Choosing the Value of K

- Small value of K → Model is sensitive to noise (high variance).
- Large value of K → Model is smoother but may miss small patterns (high bias).
- Typically, K is chosen using cross-validation.
- Use odd value of K for binary classification to avoid ties.

5. Advantages and Disadvantages

Advantages	Disadvantages
• Simple and easy to understand. • No training phase required. • Works well for small datasets. • Can be used for both classification and regression.	• Computationally expensive for large datasets. • Performance degrades with high dimensional data (Curse of Dimensionality). • Sensitive to irrelevant features and different scales. • Requires choosing the value of K.

LINEAR REGRESSION

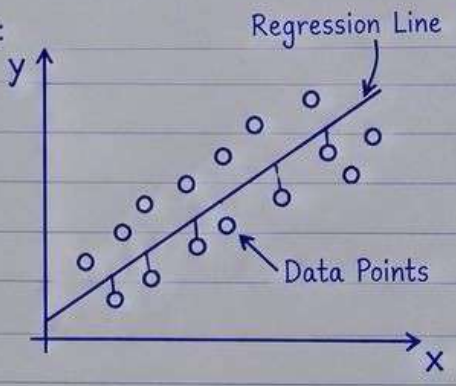
1. What is Linear Regression ?

- Linear Regression is a supervised learning algorithm used for predicting a continuous numerical value.
- It models the relationship between input features (X) and a continuous target variable (y) using a linear equation.
- It assumes a linear relationship between the independent and dependent variables.
- It is one of the simplest and most widely used regression algorithms.

2. Linear Regression Model

The equation of a simple linear regression model is :

$$y = mx + b$$



- y : Predicted output (dependent variable)
- x : Input feature (independent variable)
- m : Slope (coefficient)
- b : Intercept (bias)

3. Cost Function (Mean Squared Error)

- Linear Regression tries to find the best values of m and b by minimizing the Mean Squared Error (MSE).

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

n : Number of training examples
y_i : Actual value
$\hat{y}_i$: Predicted value
$(y_i - \hat{y}_i)^2$: Squared error

4. Parameter Estimation

- Parameters (m and b) can be learned using :
  - Normal Equation (analytical solution)
  - Gradient Descent (iterative optimization)

5. Types of Linear Regression

Type	Description	Use Case
Simple Linear Regression	One independent variable	Predict house price based on area
Multiple Linear Regression	Multiple independent variables	Predict salary based on experience, skills, etc.

6. Evaluation Metrics

- Mean Squared Error (MSE) : Measures average squared difference.
- Root Mean Squared Error (RMSE) : $\sqrt{MSE}$ (easier to interpret).
- R^2 Score : Shows how well the model explains the variance in data (0 to 1).
- Mean Absolute Error (MAE) : Average of absolute errors.

LOGISTIC REGRESSION

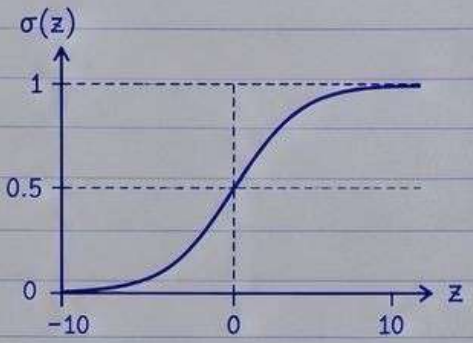
1. What is Logistic Regression ?

- Logistic Regression is a supervised learning algorithm used for classification problems.
- It predicts the probability that an input belongs to a particular class (usually binary classification).
- It uses the logistic (sigmoid) function to map any real-valued number to a value between 0 and 1.
- Despite its name, it is a classification algorithm, not a regression algorithm.

2. Logistic Function (Sigmoid)

- Logistic Regression uses the sigmoid function to convert the linear equation output into a probability.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- $z = wx + b$ (linear combination)
- Output is always between 0 and 1.

3. Hypothesis Function

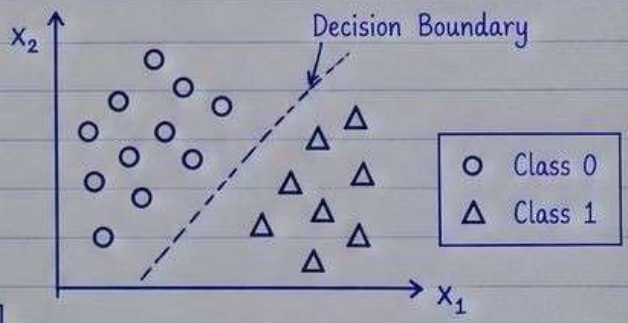
- The hypothesis function for logistic regression is :

$$h(x) = \sigma(wx + b) = \frac{1}{1 + e^{-(wx+b)}}$$

- $h(x)$ gives the probability that the input belongs to class 1.

4. Decision Boundary

- We use a threshold (usually 0.5) to decide the class :
  - $h(x) \geq 0.5 \rightarrow$ Class 1
  - $h(x) < 0.5 \rightarrow$ Class 0



5. Cost Function (Binary Cross Entropy)

- Logistic Regression uses Binary Cross Entropy (Log Loss) as the cost function.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i)) \right]$$

m : Number of training examples
y_i : Actual label (0 or 1)
$h(x_i)$: Predicted probability
θ : Model parameters (w, b)

6. Advantages and Disadvantages

Advantages	Disadvantages
• Simple and easy to implement. • Works well for binary classification. • Outputs probability (probabilistic interpretation). • Less prone to overfitting compared to complex models.	• Assumes a linear relationship between features and log-odds. • Performance may be poor with non-linear data. • Sensitive to outliers. • Not suitable for very complex datasets.

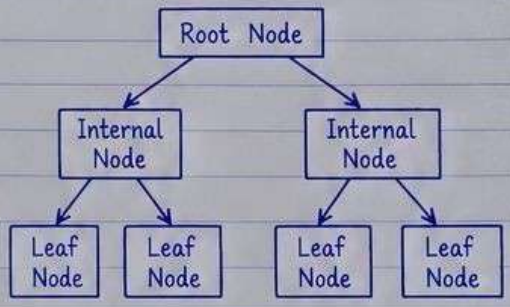
DECISION TREE

1. What is Decision Tree ?

- Decision Tree is a supervised learning algorithm used for both classification and regression.
- It splits the data into subsets based on the value of input features.
- It works by asking a series of questions on features and creating a tree like structure.
- It is easy to understand, interpret and visualize.
- It can handle both numerical and categorical data.

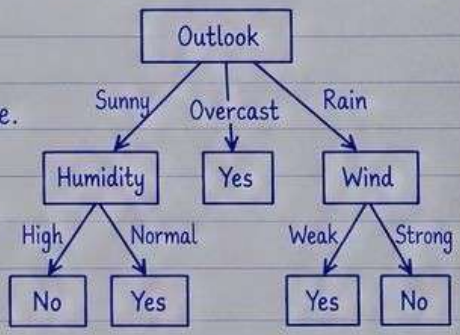
2. Structure of a Decision Tree

- Root Node : Represents the entire dataset.
- Internal Node : Decision based on a feature.
- Branch : Outcome of a decision (feature value).
- Leaf Node : Final prediction (class label or value).



3. How Decision Tree Works ?

1. Select the best feature to split the data.
2. Split the dataset into subsets based on the feature value.
3. Repeat the process recursively for each subset.
4. Stop when a stopping condition is met (e.g. max depth, minimum samples, or pure node).
5. Make prediction using the leaf node.



4. Feature Selection Criteria

- For Classification : Information Gain (Entropy) or Gini Index.
- For Regression : Mean Squared Error (MSE) or Mean Absolute Error (MAE).

5. Entropy and Information Gain (Classification)

- Entropy measures the impurity in a dataset.
- It is 0 when all samples belong to the same class.

$$\text{Entropy}(S) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

p_i : Proportion of class i
n : Number of classes

- Information Gain measures the reduction in entropy after a split.

$$IG(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

S : Original dataset
A : Feature
S_v : Subset for value v

6. Gini Index (Alternative to Entropy)

- Gini Index also measures impurity in a dataset.
- It is faster to compute than entropy.

$$\text{Gini}(S) = 1 - \sum_{i=1}^{n} p_i^2$$

p_i : Proportion of class i
n : Number of classes

7. Advantages and Disadvantages

Advantages	Disadvantages
• Easy to understand and interpret. • Works for both classification and regression. • Can handle numerical and categorical data. • No need for feature scaling. • Can capture non-linear relationships.	• Prone to overfitting (especially with deep trees). • Can be unstable (small change in data can change tree). • May not perform well on very complex datasets. • Bias towards features with more categories. • Usually lower accuracy compared to ensemble methods (Random Forest, Gradient Boosting).

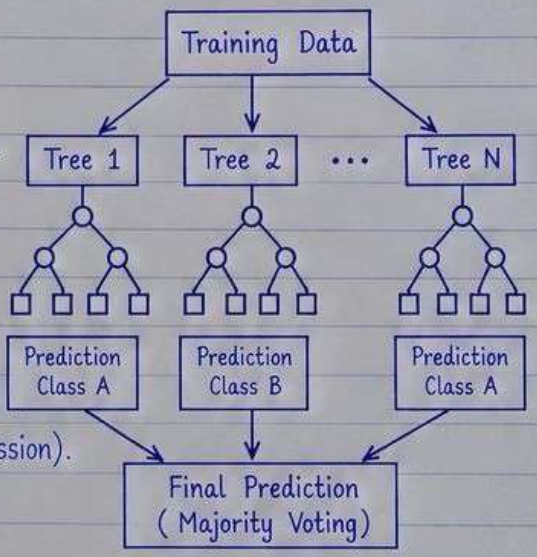
RANDOM FOREST

1. What is Random Forest ?

- Random Forest is an ensemble learning algorithm that uses multiple decision trees to solve classification and regression problems.
- It builds many decision trees on different subsets of data and features and combines their predictions to get a more accurate and stable result.
- It reduces overfitting compared to a single decision tree.
- It works well with both numerical and categorical data.
- It is easy to implement and gives good performance in most real-world problems.

2. How Random Forest Works ?

1. Take multiple random samples (with replacement) from the original training data (Bootstrap Sampling).
2. For each sample, build a decision tree.
3. At each split, consider only a random subset of features instead of all features.
4. Train all trees independently.
5. For prediction, combine the results of all trees (majority voting for classification or average for regression).



3. Prediction

- Classification : Use majority voting (most common class).
- Regression : Use the average of all trees' predictions.

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \ldots, h_n(x)\}$$

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} h_i(x)$$

4. Key Parameters

Parameter	Description	Default (sklearn)
n_estimators	Number of trees in the forest	100
max_depth	Maximum depth of each tree	None
max_features	Number of features to consider at each split	sqrt (classification)
min_samples_split	Minimum samples required to split a node	2
min_samples_leaf	Minimum samples required at a leaf node	1
bootstrap	Whether to use bootstrap sampling	True

5. Advantages and Disadvantages

Advantages	Disadvantages
• High accuracy and less overfitting. • Works well with both numerical and categorical data. • Handles large datasets. • Robust to noise and outliers. • Provides feature importance. • Easy to implement and parallelize.	• Computationally expensive (many trees). • Less interpretable than a single decision tree. • Requires more memory. • Training time can be high for large datasets. • Not ideal for real-time prediction with very limited resources.

6. Use Cases

- Classification : Spam detection, disease prediction, customer churn, fraud detection.
- Regression : House price prediction, sales forecasting, demand prediction.

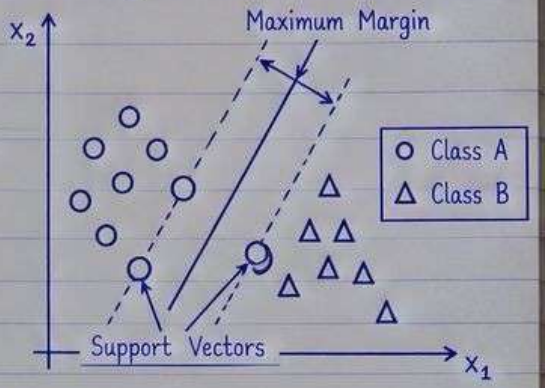
SUPPORT VECTOR MACHINE (SVM)

1. What is SVM ?

- Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression.
- It finds the optimal hyperplane that separates the classes with maximum margin.
- It works well in high dimensional spaces.
- It is effective in cases where the number of features is greater than the number of samples.
- It can be used for both linear and non-linear problems using kernel functions.

2. Optimal Hyperplane

- SVM tries to find the hyperplane that maximizes the margin between two classes.
- The data points closest to the hyperplane are called Support Vectors.
- Only support vectors affect the position of the hyperplane.
- A larger margin leads to better generalization.



3. Mathematical Formulation (Linear SVM)

For linearly separable data, the optimal hyperplane is given by:

$$w \cdot x + b = 0$$

- w : Weight vector (normal to hyperplane)
- x : Input feature vector
- b : Bias term

We maximize the margin, which is equivalent to minimizing :

$$\frac{1}{2}\|w\|^2$$

Subject to : $y_i (w \cdot x_i + b) \geq 1$ for all training samples.

4. Kernel Trick (Non-Linear SVM)

- When data is not linearly separable, SVM uses kernel functions to map the input features to a higher dimensional space where a linear separation is possible.
- Common kernel functions :

Kernel	Formula	Use Case
Linear	$K(x, x') = x \cdot x'$	Linear separable data
Polynomial	$K(x, x') = (x \cdot x' + c)^d$	Non-linear data (medium complexity)
RBF (Gaussian)	$K(x, x') = \exp(-\gamma \|x - x'\|^2)$	Most common, works well in many cases
Sigmoid	$K(x, x') = \tanh(\alpha x \cdot x' + c)$	Neural network like problems

5. Key Parameters

Parameter	Description	Default (sklearn)
C	Regularization parameter (trade-off between margin and misclassification)	1.0
kernel	Type of kernel function	'rbf'
gamma	Kernel coefficient (for rbf, poly, sigmoid)	'scale'
degree	Degree for polynomial kernel	3
coef0	Independent term in kernel function	0.0

6. Advantages and Disadvantages

Advantages	Disadvantages
• Works well in high dimensional spaces.	• Computationally expensive for large datasets.
• Effective in both linear and non-linear cases.	• Not suitable for very large datasets.
• Uses only support vectors (memory efficient).	• Sensitive to the choice of kernel and parameters.
• Robust to overfitting with proper regularization.	• Less interpretable compared to decision trees.

7. Use Cases

- Classification : Image classification, text categorization, spam detection, disease prediction.
- Regression : House price prediction, demand forecasting (using SVR).

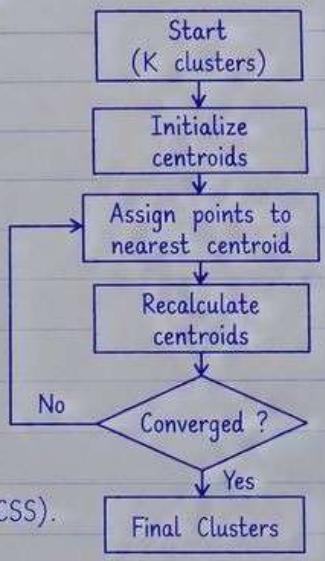
K-MEANS CLUSTERING

1. What is K-Means Clustering ?

- K-Means is an unsupervised machine learning algorithm used for clustering.
- It groups similar data points into K different clusters based on feature similarity.
- It aims to minimize the distance between data points and their respective cluster centroids.
- Each data point belongs to the cluster with the nearest centroid.
- It is simple, fast and widely used for real-world problems.

2. How K-Means Works ?

1. Choose the number of clusters (K).
2. Randomly initialize K centroids.
3. Assign each data point to the nearest centroid.
4. Recaclulate the centroids as the mean of all points in each cluster.
5. Repeat steps 3 and 4 until centroids no longer change or a maximum number of iterations is reached.



3. Objective Function

- K-Means tries to minimize the Within-Cluster Sum of Squares (WCSS).
- The objective is to minimize the distance between data points and their cluster centroid.

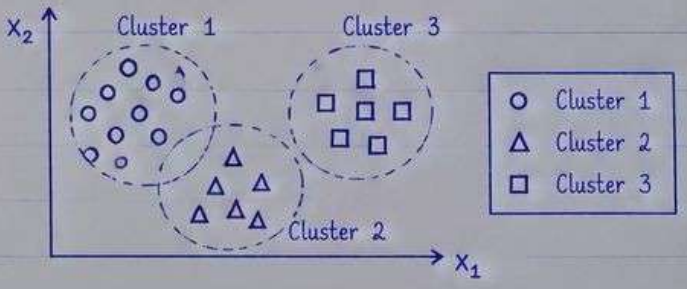
$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \| x - \mu_i \|^2$$

- K : Number of clusters
- C_i : Set of points in cluster i
- x : Data point
- μ_i : Centroid of cluster i
- $\| x - \mu_i \|^2$: Squared Euclidean distance

4. Distance Metric

- K-Means usually uses Euclidean distance.

$$d(x, \mu) = \sqrt{\sum_{i=1}^{n} (x_i - \mu_i)^2}$$



5. Key Parameters

Parameter	Description	Default (sklearn)
n_clusters	Number of clusters (K)	8
init	Method for centroid initialization (e.g., k-means++, random)	'k-means++'
n_init	Number of times the algorithm is run with different initial centroids	'auto'
max_iter	Maximum number of iterations	300
random_state	Controls randomness for reproducibility	None

6. Advantages and Disadvantages

Advantages	Disadvantages
• Simple and easy to implement.	• Need to specify the number of clusters (K).
• Works well on large datasets.	• Sensitive to initial centroid selection.
• Computationally efficient.	• Does not work well with non-spherical clusters.
• Good for spherical and well-separated clusters.	• Sensitive to outliers.

7. Use Cases

- Customer segmentation, image compression, document clustering, market analysis.
- Anomaly detection (as a preprocensing step), grouping similar products.

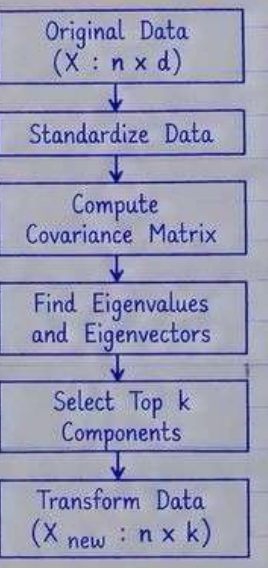
PRINCIPAL COMPONENT ANALYSIS (PCA)

1. What is PCA ?

- Principal Component Analysis (PCA) is an unsupervised learning technique used for dimensionality reduction.
- It transforms the original correlated features into a new set of uncorrelated features called principal components.
- The principal components capture the maximum variance from the data.
- It helps to reduce the dimensionality while retaining the most important information.
- PCA is widely used for visualization, noise reduction and improving model performance.

2. Why Use PCA ?

- Reduces the number of features (dimensionality).
- Removes correlation between features.
- Helps in visualizing high-dimensional data (e.g., 2D or 3D).
- Reduces noise and can improve model performance.
- Speeds up training time by reducing computational complexity.



3. How PCA Works ?

1. Standardize the data (mean 0, unit variance).
2. Compute the covariance matrix.
3. Calculate eigenvalues and eigenvectors.
4. Sort eigenvectors by eigenvalues (descending order).
5. Select top k eigenvectors (principal components).
6. Project the original data onto the new feature space.

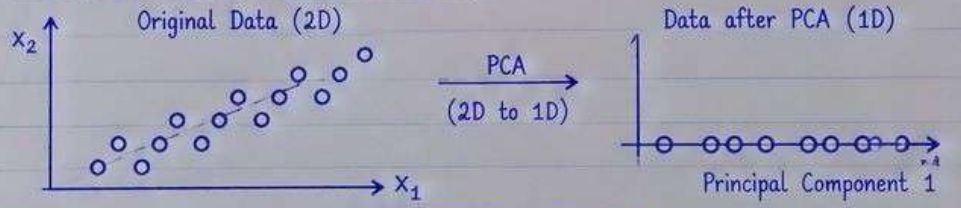
4. Mathematical Formulation

- Given a dataset X (n samples, d features):

1. Standardize the data : $Z = (X - \mu) \quad \sigma$
2. Covariance matrix : $C = \frac{1}{n-1} Z^T Z$
3. Eigen decomposition : $C\, v = \lambda\, v$
4. Transform data : $X_{new} = Z \cdot W_k$

λ : Eigenvalue (amount of variance)
v : Eigenvector (principal component)
W_k : Matrix of top k eigenvectors

5. Example (2D to 1D Projection)



The new axis (Principal Component 1) captures the maximum variance in the data.

6. Key Parameters (sklearn.decomposition.PCA)

Parameter	Description	Default
n_components	Number of principal components to keep	None
whiten	Whether to apply whitening (remove correlations)	False
svd_solver	SVD solver to use (auto, full, randomized)	'auto'
random_state	Controls randomness for reproducibility	None

7. Advantages and Disadvantages

Advantages	Disadvantages
• Reduces dimensionality and computation time. • Removes feature correlation. • Helps in visualization. • Can improve model performance. • Reduces noise in the data.	• Results can be hard to interpret. • May lose some information. • Assumes linear relationships. • Sensitive to feature scaling. • Not suitable for non-linear data (use t-SNE/UMAP).

8. Use Cases

- Image compression, data visualization, gene expression analysis, customer segmentation, noise reduction.